

Memory match

Overview

The first "grid" game of the book, built three times. The Phaser content is light - sprite sheet frames on an `Image`, `setFrame`, tweens and tween chains, `delayedCall`, a `Container`, particles - and almost all of it was met in earlier chapters. The real subject is **program structure**: laying out a grid with arithmetic rather than typed-in positions, writing game rules as a **state machine**, and waiting for animations with **callbacks**. The advanced version adds an OO design discussion that suits Java programmers well: a `Theme` interface with two implementations, and a `TileFace` that separates what a tile *shows* from what it *matches*.

The one idea to get across above all others: **the board must be locked while tiles are showing**, and a single `state` field is the cleanest way to do it. Students who "fix" the three-tiles-up bug with a growing collection of booleans are the ones to talk to.

Prerequisites

- Chapter 2: scenes, `init(data)` and resetting fields, `setInteractive` and `"pointerdown"`, the registry
- Chapter 5 (sounds), Chapter 6 (local storage, validating what is read back), Chapter 7 (tweens, chains, stagger, particles)
- Chapter 12: the `Card / Deck` split, Fisher-Yates, a flip made of two `scaleX` tweens, game state as a union of string literals, and the `cards.png` frame sum
- Java interfaces and lambdas (`Runnable`) for the analogies

Learning outcomes

Students can:

1. lay out an $R \times C$ grid of game objects with nested loops, centred in the game, and convert between (row, col) and a flat index with `row * COLS + col`

2. make shuffled pairs (or groups) of values with `Phaser.Utils.Array.Shuffle`
3. show and change the frame of a sprite sheet on an `Image`
4. describe a game's rules as a state machine (states, events, transitions), implement it with a union type, and use it to ignore input in "locked" states
5. animate a flip with two tweens, and run code when it finishes by passing a callback
6. turn a performance (moves, time) into a rating, and show it
7. (advanced) design an interface that lets the same game use different sets of content, and describe levels as data

Suggested session plan (2 x 2 hours)

Session 1 - the simple and intermediate versions

Time	Activity
0:00 - 0:10	Play a physical round of pairs with a dozen cards, if you have some. Ask: "what are the rules, exactly? What happens if I turn over a third card while two are showing?"
0:10 - 0:30	Slides 3-6: what makes the genre work, tiles as sprite sheet frames, making pairs, the grid maths. Students draw the 4 x 3 grid on paper and work out the centre of tile 7 before seeing <code>layoutGrid</code>
0:30 - 0:50	Slides 7-9: the state machine, and trying the simple version. Live demo: delete <code>this.state = "twoUp"</code> in the simple version and click three tiles quickly - see "Common problems" below for the crash. Discuss why
0:50 - 1:20	Challenges 1 and 5 (both on the simple version)
1:20 - 1:40	Slides 10-13: the flip, callbacks, the fourth state, stars. Live-code the flip from scratch in a blank tile (one tween first, then the second in <code>onComplete</code>)
1:40 - 2:00	Start challenge 2 or 3

Session 2 - the advanced version

Time	Activity
0:00 - 0:15	Play the advanced version, both themes. Ask: "what had to change so that two <i>different</i> cards can match?"
0:15 - 0:40	Slides 14-15: levels as data, the board that fits, the <code>Theme</code> interface and <code>matchKey</code> . Design discussion (below)
0:40 - 0:55	Slides 16-18: peek and shuffle as two more locked states; buttons and best results
0:55 - 2:00	Challenges 2, 3, 4 and 6

Key points to stress

- **Positions are calculated, not typed.** A grid that centres itself from `ROWS` , `COLS` , `GAP` and the tile size can change size without anyone touching the layout code - which is exactly what challenge 1 and the advanced levels rely on
- **Gaps are one fewer than tiles.** `COLS * TILE_SIZE + (COLS - 1) * GAP` . The off-by-one here only shows as a grid slightly off-centre, so students often do not notice it
- **One `state` field, not several booleans.** A state machine can only be in one state at once, so impossible combinations cannot happen. Each new feature (a flip, a peek, a shuffle, a hint, a time limit) is a new state and a few new transitions
- **Test what is allowed, not what is forbidden.** The intermediate version locks with `state === "twoUp" || state === "checking"` ; the advanced one with `state !== "idle" && state !== "oneUp"` . Only the second stays correct when a new locked state is added. Challenge 3 walks straight into this (see solutions.md)
- **Animations take time, so logic must wait for them.** The pair is checked in the second flip's callback, not straight after starting it; the board unlocks in the callback of the flip back
- **The tile knows what it is; the scene knows the rules.** `Tile` has `flipUp` , `celebrate` , `shake` , `matches` - nothing about pairs, moves or turns. That is why challenges 4 and 5 barely touch `Tile`
- **Shape and meaning are different things.** `frame` vs `matchKey` in the advanced version - the same idea as Chapter 12's model/view split

Common problems and errors

What students see	Cause	Fix
Three (or four) tiles face up at once, then <code>TypeError: Cannot read properties of null (reading 'matches')</code> in the console	the lock is missing: after a wrong pair the state stayed <code>"oneUp"</code> with <code>firstTile</code> already <code>null</code> (produced by deleting <code>this.state = "twoUp"</code> from the simple version)	put the game in a locked state when wrong pair shows
Argument of type <code>'Tile null'</code> is not assignable to parameter of type <code>'Tile'</code> . Type <code>'null'</code> is not assignable to type <code>'Tile'</code> .	passing <code>this.firstTile</code> (a <code>Tile null</code>) where a <code>Tile</code> is needed	the state guarantees it is set: <code>this.firstTile!</code> - or check for
Type <code>"oneup"</code> is not assignable to type <code>'State'</code> . Did you mean <code>"oneUp"</code> ?	a typo in a state name	this is why the states are a union type, not plain strings
The game restarts the moment the last pair is found	<code>this.input.once("pointerdown", ...)</code> added inside the tile's click handler: the same click reaches the scene's listener straight after (tested: the scene restarts on the winning click)	use a key, a <code>delayedCall</code> , or a separate win scene
Second game cannot be won; or tiles from the first game are still "there"	<code>tiles</code> (or <code>pairsFound</code> , <code>state</code>) not reset in <code>init()</code> ; the array still holds the old, destroyed tiles	reset every field in <code>init()</code> - Chapter 2's trap
Grid is a little off-centre	gaps counted as <code>COLS * GAP</code> instead of <code>(COLS - 1) * GAP</code> , or the half-tile for the centre origin forgotten	check the sums against the diagram

What students see	Cause	Fix
The result sound plays before the second tile can be seen	the pair is checked straight after <code>flipUp()</code> starts, not in its callback	pass <code>() => this.checkPair()</code> to <code>flipUp</code>
After a flip, a scaled-down tile is suddenly full size	the flip tweens <code>scaleX</code> back to <code>1</code>	tween back to the tile's own base (advanced <code>Tile</code>)
A dark tile tinted yellow looks darker, not brighter	<code>setTint</code> multiplies colours	<code>setTintMode(Phaser.TintModes</code> (challenge 3's solution), or tween <code>alpha</code> / <code>scale</code> instead
Class <code>'CardTheme'</code> incorrectly implements interface <code>'Theme'</code> . Property <code>'backFrame'</code> is missing in type <code>'CardTheme'</code> but required in type <code>'Theme'</code> .	a class that implements <code>Theme</code> has left a property out	the interface doing its job - add it
Property <code>'cards'</code> is missing in type <code>'{ pictures: PictureTheme; }'</code> but required in type <code>'Record<ThemeName, Theme>'</code> .	a name added to <code>ThemeName</code> with no theme in <code>THEMES</code> (or the reverse)	add the theme; <code>Record</code> insists every key has a value
No sound on the first flip	the browser blocks audio until the page has been clicked	the title scene's "Click to start" so this: mention Chapter 5

Demos and live-coding

- **Remove the lock** (simple version): delete `this.state = "twoUp";` in `checkPair`, rebuild, and click three tiles quickly. Three tiles stay up and the console shows the `TypeError` above. Ask the class to trace, state by state, what happened

- **The flip from nothing:** in a blank scene, one tile. Tween `scaleX` to 0 - "now it has vanished, what next?" - add `onComplete` with `setFrame`, then the second tween. Then change the eases to `"Linear"` and compare
- **The easing matters:** in the intermediate project, set `HALF_FLIP` to 600 and watch the flip in slow motion; then try `"Back.easeOut"` on the second half
- **The trap of `once("pointerdown")`:** in the simple version, change `win()` to use `this.input.once("pointerdown", ...)`. Find the last pair - the game restarts instantly. Explain the order Phaser delivers the event (game object first, then the scene)

Discussion questions

- Draw the state machine for the simple version on the board. Which transitions does each click cause? Which state has no transition on a click, and why is that the important one?
- Why is `face` `readonly` on a `Tile`? What could go wrong if the scene could change it?
- The intermediate version checks the pair in a callback. What would players notice if it checked straight away? (The sound and shake come before the second face is visible.)
- In the card theme, the seven of hearts matches the seven of diamonds. Why could the simple version's `Tile` (with `face: number`) not express that? What is the smallest change that could?
- `LEVELS` is an array of plain objects. What would change if each level were a subclass of a `Level` class instead? When would that be worth it? (When levels have different *behaviour*, not just different numbers.)
- Is the peek worth a star? How would you tune the star limits - by feel, or by recording real players' moves?

Extension ideas

- a "flip everything face up for two seconds at the start" memorisation phase, as a new state
- a third theme - letters, numbers, or shapes drawn with `Graphics` and `generateTexture`
- matching by a *rule* rather than a picture: a number and a sum that makes it (7 and "3 + 4")

- a replay: record every click (time and tile) in an array, then play the game back
 - a proper undo of the last pair? (Why does that not make sense for this genre?)
-

Assessment ideas

- Practical: "add a fourth theme with its own `makeFaces` rule" (tests the interface), or "make the wrong-pair delay get shorter as more pairs are found" (tests the state machine and timers)
 - Code reading: give the intermediate `tileClicked` and `checkPair` without comments, and a list of click times; ask which clicks are ignored
 - Short answer: explain why the pair is checked in `flipUp`'s callback, and what a callback is, using a Java comparison
 - Design: draw the state diagram for challenge 4 (two players) or challenge 5 (triples)
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment. Every solution was played with the test harness; what was checked is noted under each.

1. Choose the size

Project: `solutions/ch13_challenge_1_choose_the_size` (from `ch13_memory_simple`)

Goal: turn constants into state that survives a restart - `scene.restart(data)` and `init(data)`.

`ROWS` and `COLS` become fields, and the three sizes a table:

`src/scenes/GameScene.ts`

```
const SIZES: BoardSize[] = [
  { rows: 3, cols: 4 },
  { rows: 4, cols: 4 },
  { rows: 4, cols: 6 },
];
...
export interface BoardSize {
  rows: number;
  cols: number;
}
...
private rows = SIZES[0].rows; // CHALLENGE 1: the size is a field now, not
a constant
private cols = SIZES[0].cols;
```

The keys restart the scene with a size; `init` takes it, or keeps the size it had:

```
init(data: Partial<BoardSize>): void {
  this.rows = data.rows ?? this.rows;
  this.cols = data.cols ?? this.cols;
  ...
  this.input.keyboard!.on("keydown-ONE", () => {
    this.scene.restart(SIZES[0]);
  });
}
```

- the first start (from the game config) and `restart()` with no data both hand `init` an **empty object**, so the fields may be missing. `Partial<BoardSize>` is "a `BoardSize` with every field optional", and `??` (the nullish coalescing operator) falls back to the current size when a field is missing. So SPACE after a win - `this.scene.restart()`, unchanged - plays again at the same size, for free
- `layOutGrid` swaps `ROWS / COLS` for `this.rows / this.cols`; nothing else in it changes, because it was already centring by arithmetic. That is the point of the challenge

Tested: 1, 2, 3 give 12, 16 and 24 tiles; the 6 x 4 grid spans x 60-740 and y 76-524 (centred); SPACE after winning the 4 x 4 board gives another 4 x 4 board.

Look for: the size in the restart data (not a field set before `restart`, which works but by accident - the scene object is reused); the grid still centred at every size; the message saying the size. Students who keep the size in the **registry** instead have a good answer too. Some will ask about a 5 x 5 board: 25 tiles cannot make pairs - a nice moment to add a check that `rows * cols` is even.

2. Against the clock

Project: `solutions/ch13_challenge_2_against_the_clock` (from `ch13_memory_intermediate`)

Goal: a losing condition - and the discovery that it can happen in the *middle* of something.

The countdown is the same clock, shown the other way round:

```
src/scenes/GameScene.ts
```

```

if (this.seconds >= TIME_LIMIT) {
  this.seconds = TIME_LIMIT;
  this.outOfTime();
}
...
const left = TIME_LIMIT - this.seconds;
this.timeText.setText(`Time left: ${left.toFixed(1)}`);
this.timeText.setColor(left <= HURRY_TIME ? "#e63946" : "#ffffff");

```

Running out of time is a new, permanently locked state, "timeUp" :

```

private outOfTime(): void {
  this.clockRunning = false;
  this.state = "timeUp";

  // cancel a wrong pair that is waiting to turn back (the tiles simply stay
face up)
  this.time.removeAllEvents();

  // anything face down - or on its way down - turns face up
  for (const tile of this.tiles) {
    if (!tile.isFaceUp()) {
      tile.reveal();
    }
  }
}

```

The hard part is that time can run out at any moment: while the second tile is flipping up, while a wrong pair is waiting to turn back, or while it is half-way through turning back. Three small pieces cover all of them:

- `this.time.removeAllEvents()` cancels the pending "turn the wrong pair back" call
- `checkPair` starts with `if (this.state === "timeUp") { return; }`, so a flip that finishes late does nothing
- `Tile.reveal()` stops any flip in progress - and with it the callback that would have unlocked the board - and flips the tile up:

`src/objects/Tile.ts`

```
public reveal(): void {
    this.scene.tweens.killTweensOf(this);
    this.scaleX = 1;
    this.flipUp();
}
```

`WinData` gains `pairsFound` and `outOfTime`; the win scene shows "Out of time!", no earned stars (all three grey) and "Pairs found: 1 of 8", and plays the wrong sound instead of the win sound.

Tested: time forced to run out (by moving `startTime` back) while a wrong pair was half-way through turning back: every tile ends face up on its own face with `scaleX` 1, the state stays `"timeUp"`, and the win scene reports `outOfTime: true`. The time turns red below ten seconds.

Look for: the timer counting down from the first click (not from the scene starting - either is defensible, but be consistent with the chapter); a locked state for the end; and whether the student thought about the "in the middle of a flip" cases. Many solutions work only when time runs out while the board is idle - play one with a wrong pair showing at 0.5 seconds left and see. A new scene for losing is also a fine answer.

3. A hint

Project: `solutions/ch13_challenge_3_a_hint` (from `ch13_memory_intermediate`)

Goal: another locked state - and a lesson in how to write a lock.

`src/scenes/GameScene.ts`

```

private hint(): void {
  if (this.state !== "idle") {
    return;
  }
  const hidden = this.tiles.filter((tile) => !tile.isFaceUp());
  if (hidden.length === 0) {
    return;
  }
  const face = Phaser.Utls.Array.GetRandom(hidden).face;
  const pair = hidden.filter((tile) => tile.face === face);

  this.state = "hinting";
  this.moves = this.moves + HINT_COST;
  this.updateHud();
  for (const tile of pair) {
    tile.flash(() => {
      this.state = "idle";
    });
  }
}

```

In "idle" no unmatched tile is face up, so "face down" means "not yet found" - pick one at random, and flash it and its partner. `Tile.flash` pulses the tile three times and glows it yellow:

src/objects/Tile.ts

```

public flash(onDone: () => void): void {
  this.setTint(HINT_TINT).setTintMode(Phaser.TintModes.ADD);
  this.scene tweens.add({
    targets: this,
    scale: 1.12,
    duration: 150,
    yoyo: true,
    repeat: 2,
    onComplete: () => {
      this.clearTint().setTintMode(Phaser.TintModes.MULTIPLY);
      onDone();
    },
  });
}

```

The ordinary tint *multiplies* colours, which can only make the dark tile darker; `ADD` brightens it. (Both tiles call `onDone`, so the state is set to `"idle"` twice - harmless.)

The bug the solution walked into. The first version of this solution added `"hinting"` to the `State` type and left `tileClicked` as it was:

```
if (this.state === "twoUp" || this.state === "checking") {
```

Clicking a tile during the hint then fell through to the "second tile" branch, with no first tile, and the harness showed `TypeError: Cannot read properties of null (reading 'matches')`. The fix is to list the states that **allow** a click, as the advanced version does:

```
if (this.state !== "idle" && this.state !== "oneUp") {  
  return;  
}
```

Now any state added later is locked unless someone decides otherwise. This is the most useful thing to discuss from this challenge.

Tested: H in `"idle"` flashes two tiles with the same face and adds 2 moves; clicks during the flash are ignored; H with one tile up does nothing.

Look for: the hint only when idle; the moves added; a new state or an equivalent lock while the flash plays (without one, a click starts a flip - a `scaleX` tween - on a tile that is also being scaled by the flash, and the two fight). Flipping the pair face up briefly is also an acceptable "flash".

4. Two players

Project: `solutions/ch13_challenge_4_two_players` (from `ch13_memory_intermediate`)

Goal: find the one place in the state machine where a turn ends.

`src/scenes/GameScene.ts`

```

private player = 0;
private scores = [0, 0];
...
this.scores[this.player] = this.scores[this.player] + 1;
this.updateHud();
...
second.flipDown(() => {
    // CHALLENGE 4: THE place where a turn ends - a wrong pair, now face down
    again
    this.player = 1 - this.player;
    this.updateHud();
    this.state = "idle";    // unlock only once both are face down again
});

```

A pair scores for the current player, who simply goes again - nothing to do. A wrong pair passes the turn, and the right moment is when the tiles are face down and the board unlocks. Switching when the wrong pair is *found* would show "Player 2's turn" while player 1's tiles are still up. `1 - this.player` flips between 0 and 1.

The HUD shows both scores and whose turn it is, in that player's colour. `WinData` carries `scores`, and the win scene announces the winner or a draw:

`src/scenes/WinScene.ts`

```

const [p1, p2] = this.result.scores;
let heading = "A draw!";
if (p1 > p2) {
    heading = "Player 1 wins!";
} else if (p2 > p1) {
    heading = "Player 2 wins!";
}

```

`const [p1, p2] = ...` is **destructuring**: the first two items of the array, as two constants.

Stars are dropped: moves are shared by both players, so they no longer say how well either played.

Tested: player 1 finds a pair and keeps the turn (score 1-0); a wrong pair passes to player 2, who finds three (1-3); a wrong pair passes back; the game ends 5-3 and the win scene says "Player 1 wins!". Playing again starts at 0-0, player 1.

Look for: the turn switched in exactly one place; scores reset in `init()` - and reset with a **new** array (`this.scores = [0, 0]`), because the old array was handed to the win scene inside `WinData` , and changing it in place would change the result being shown. Worth asking students why that matters even though the win scene has already drawn it.

5. Triple match

Project: `solutions/ch13_challenge_5_triple_match` (from `ch13_memory_simple`)

Goal: generalise from "a first tile" to "the tiles showing", and redesign the states.

`src/scenes/GameScene.ts`

```
type State = "idle" | "picking" | "wrong";    // CHALLENGE 5
...
private showing: Tile[] = [];
```

```
tile.showFace();
this.showing.push(tile);

if (!tile.matches(this.showing[0])) {
  // a different face - this group is wrong, however many are showing
  this.state = "wrong";
  const wrongGroup = this.showing;
  this.showing = [];
  this.time.delayedCall(MISMATCH_DELAY, () => {
    for (const wrongTile of wrongGroup) {
      wrongTile.hideFace();
    }
    this.state = "idle";
  });
} else if (this.showing.length === GROUP_SIZE) {
```

- `makeFaces` pushes each face `GROUP_SIZE` times; the board holds `ROWS * COLS / GROUP_SIZE` groups
- comparing each new tile with the **first** one is enough: if every tile matches the first, they all match each other

- `wrongGroup` keeps hold of the array for the delayed call, while `this.showing` starts afresh

With `GROUP_SIZE = 2` this is the ordinary game again - a good test that the generalisation is right. How big can the board be? Twelve faces make 36 tiles - 6 x 6, which does not fit at 100 pixels a tile (the grid would be 680 pixels tall), so it would need smaller tiles or scaling as in the advanced version.

Tested: three the same stay up; two different are wrong straight away (after two clicks); two the same and a third different is wrong with three showing; clicks during `"wrong"` are ignored; all four groups found wins.

Look for: the array rather than `firstTile / secondTile / thirdTile`; the lock still there; a wrong guess ending the group as soon as it is wrong (a student who waits for three tiles before comparing has made a different - but playable - game; ask which is better).

6. Keyboard control

Project: `solutions/ch13_challenge_6_keyboard_control` (from `ch13_memory_advanced`)

Goal: separate *where* from *what*: a cursor is a place on the board, and a tile is whatever is there now.

`layOutGrid` keeps the numbers it already worked out, so a column and row can be turned into x and y on any level, in either theme, at any scale:

`src/scenes/GameScene.ts`

```
this.gridLeft = left;
this.gridTop = top;
this.stepX = (width + GAP) * scale;
this.stepY = (height + GAP) * scale;
```

The cursor is a `Rectangle` with a stroke and no fill, drawn above the tiles, and the keys move it (clamped to the board) or pick the tile under it:

```

private moveCursor(dCol: number, dRow: number): void {
    this.cursorCol = Phaser.Math.Clamp(this.cursorCol + dCol, 0,
this.level.cols - 1);
    this.cursorRow = Phaser.Math.Clamp(this.cursorRow + dRow, 0,
this.level.rows - 1);
    this.cursor.setPosition(this.placeX(this.cursorCol),
this.placeY(this.cursorRow));
}

private pickAtCursor(): void {
    const x = this.placeX(this.cursorCol);
    const y = this.placeY(this.cursorRow);
    const tile = this.tiles.find((t) => Phaser.Math.Distance.Between(t.x, t.y,
x, y) < 1);
    if (tile !== undefined) {
        this.tileClicked(tile);
    }
}

```

`pickAtCursor` hands the tile to the same `tileClicked` the mouse uses, so every rule - the locks, face-up tiles, matched tiles - applies to the keyboard without a line of new logic. And because it looks for the tile *at the place* every time, it keeps working after a shuffle has moved the tiles round (`find` returns `undefined` if nothing is there - it never is, but the type says it could be).

Tested on level 4 with cards (6 x 4): the cursor starts on the top-left card, clamps at the edges, SPACE and ENTER turn over the card under it; after `shuffleUnmatched()` the card picked is the one now at the cursor's place.

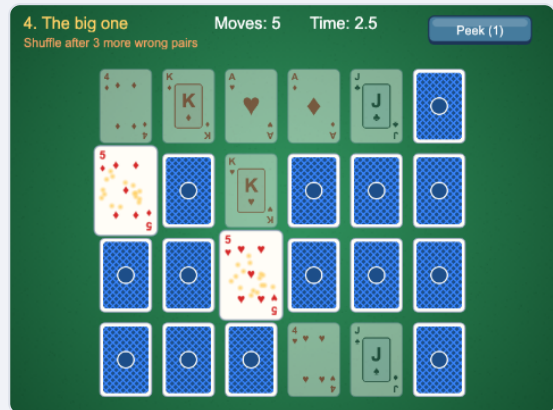
Look for: the cursor as (col, row) rather than a reference to a tile - a student who stores "the tile under the cursor" finds it wrong after the first shuffle, which is exactly the point of the hint; the tile found by position (or a 2D array of tiles kept up to date when the shuffle happens - also good, but more to keep in step); the mouse still working; the cursor sized for both themes.

Slides

Chapter 13

Memory match

Turn over two. Keep them if they match.



1 / 20

Today

- a grid laid out **by arithmetic**
- shuffled pairs, dealt onto the grid
- the rules as a **state machine** - and locking the board
- a flip made of two tweens, and **callbacks**
- moves, time and stars
- levels as data; a **Theme** interface; peek and shuffle

2 / 20

What makes a memory game work

- it is a game about **information**: every tile turned tells you something
- the player must **see** a wrong pair - about a second
- the board must be **fair**: no third tile while two are showing
- score **efficiency**: moves and time
- **feedback**: flip, sound, pop, shake

3 / 20

A tile is one frame of a sprite sheet

[image: w:900]

```
this.load.spritesheet(TILES_KEY, TILES_FILE, { frameWidth: TILE_SIZE, frameHeight: TILE_SIZE });
...
super(scene, x, y, TILES_KEY, BACK_FRAME);
...
public showFace(): void {
    this.faceUp = true;
    this.setFrame(this.face);
}
```

A **Tile** is an **Image** that starts on frame 0 (the back). An **Image** can show any frame; **Sprite** is for animation.

4 / 20

Pairs, then a grid

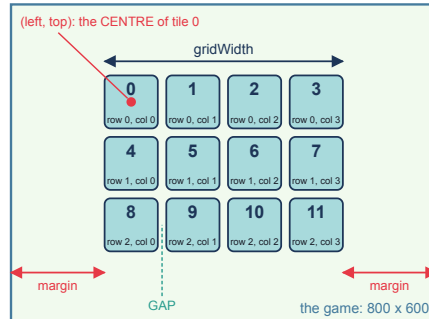
```
for (let face = 1; face <= pairs; face++) {  
  faces.push(face, face);  
}  
return Phaser.Utills.Array.Shuffle(faces);
```

```
for (let row = 0; row < ROWS; row++) {  
  for (let col = 0; col < COLS; col++) {  
    const x = left + col * (TILE_SIZE + GAP);  
    const y = top + row * (TILE_SIZE + GAP);  
    const face = faces[row * COLS + col];
```

5 / 20

Centring the grid

Laying out a 4 x 3 grid, centred in the game



The sums in layOutGrid()

```
gridWidth = COLS * TILE_SIZE + (COLS - 1) * GAP  
four tiles across, but only three gaps between them
```

```
left = (800 - gridWidth) / 2 + TILE_SIZE / 2  
two equal margins, so the grid is centred - and  
+ TILE_SIZE / 2, because a tile's origin is its centre
```

```
x = left + col * (TILE_SIZE + GAP)
```

```
y = top + row * (TILE_SIZE + GAP)  
each step across or down is one tile and one gap
```

```
face = faces[row * COLS + col]
```

the shuffled faces are dealt out row by row; the
number on each tile is its index in the array

6 / 20

The rules are a state machine

```
type State = "idle" | "oneUp" | "twoUp";

switch (this.state) {
  case "idle":
    tile.showFace();
    this.firstTile = tile;
    this.state = "oneUp";
    break;
  case "oneUp":
    tile.showFace();
    this.checkPair(this.firstTile!, tile);
    break;
  case "twoUp":
    break;
}
```

"twoUp" : a wrong pair is showing - the click is ignored. That is the **lock**

7 / 20

A wrong pair locks the board

```
} else {
  // leave the wrong pair showing long enough to remember, then turn both back
  this.state = "twoUp";
  this.time.delayedCall(MISMATCH_DELAY, () => {
    first.hideFace();
    second.hideFace();
    this.state = "idle";
  });
}
```

- one `state` field - never two states at once
- remove the lock: three tiles up, then a `TypeError`

8 / 20

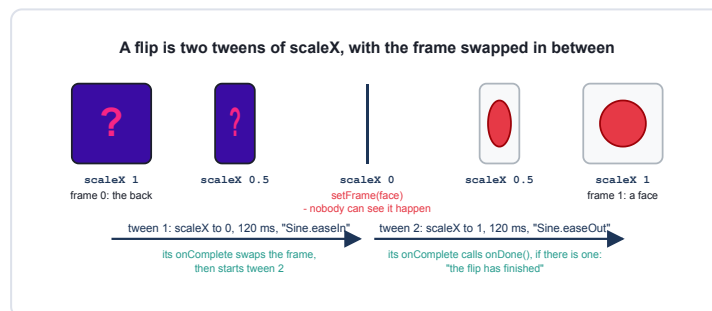
Try it: the simple version

- build, serve, play `ch13_memory_simple`
- change `ROWS`, `COLS` and `GAP` - the grid lays itself out
- what goes wrong with a 5 x 5 board?



9 / 20

The flip: two tweens of `scaleX`



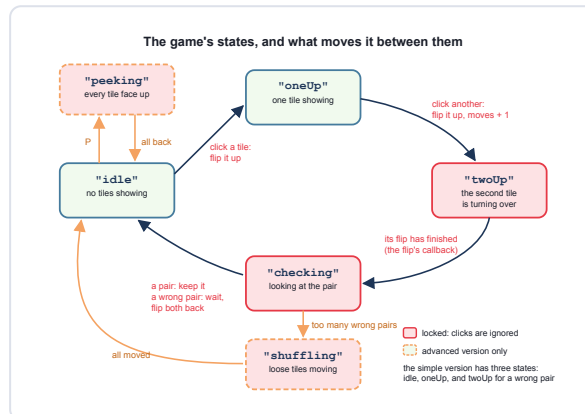
```
this.scene.tweens.add({
  targets: this,
  scaleX: 0,
  duration: HALF_FLIP,
  ease: "Sine.easeIn",
  onComplete: () => {
    this.setFrame(frame);
    this.scene.tweens.add({
      ...
```

10 / 20

Wait for it: callbacks, and a fourth state

```
public flipUp(onDone?: () => void): void {
  ...
  this.state = "twoUp";
  ...
  tile.flipUp(() => {
    this.checkPair();
  });
}
```

`onDone` is a function (like a Java `Runnable`); `?` makes it optional

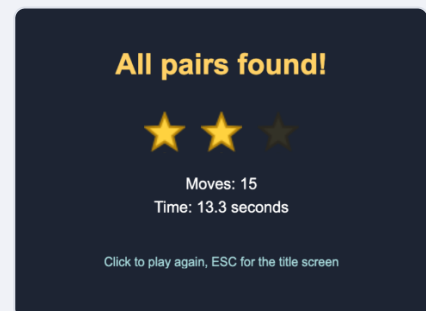


11 / 20

Moves, time and stars

```
export function starsFor(moves: number, pairs: number): number
{
  if (moves <= Math.ceil(pairs * THREE_STARS)) {
    return 3;
  }
  if (moves <= Math.ceil(pairs * TWO_STARS)) {
    return 2;
  }
  return 1;
}
```

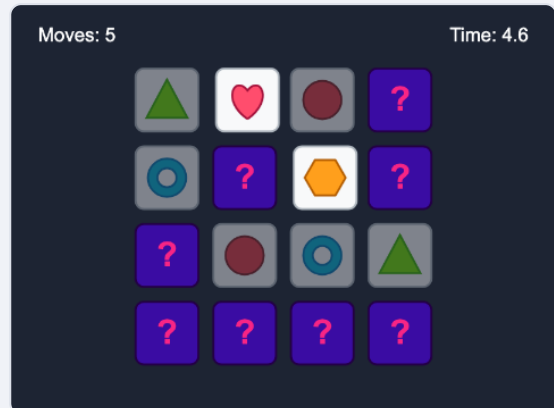
- a **move** = two tiles; the clock starts at the first click
- show the stars you *missed*, too



12 / 20

Try it: the intermediate version

- play `ch13_memory_intermediate` - how many moves for three stars?
- set `HALF_FLIP` to 600: watch the flip slowly
- try "Linear" instead of the `Sine` eases



13 / 20

Levels are data; the board fits

```
export const LEVELS: LevelData[] = [
  { name: "Warm up", cols: 4, rows: 3, shuffleAfter: 0 },
  { name: "Four square", cols: 4, rows: 4, shuffleAfter: 0 },
  { name: "Shifting sands", cols: 5, rows: 4, shuffleAfter: 4 },
  { name: "The big one", cols: 6, rows: 4, shuffleAfter: 3 },
];
```

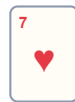
```
const scale = Math.min(1, AREA_WIDTH / gridWidth, AREA_HEIGHT / gridHeight);
```

Flips tween back to the tile's `baseScale`, not to 1.

14 / 20

Themes: what a tile shows vs what it means

Same picture, or same meaning? A TileFace has both

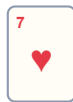


frame: 32
"7 red"



frame: 19
"7 red"

match



frame: 32
"7 red"



frame: 45
"7 black"

no match

frame
what the tile shows -
a different card for each

matchKey
what the tile means -
tiles match when these
are the same

```
export interface TileFace {  
  frame: number;      // the sprite sheet frame shown when the tile is face up  
  matchKey: string;   // two tiles match when their matchKeys are the same  
}
```

PictureTheme and CardTheme both implements Theme

15 / 20

Two more locked states

```
private peek(): void {  
  if (this.state !== "idle" || this.peeksLeft === 0) {  
    return;  
  }  
  this.state = "peeking";  
  ...  
}
```

```
const loose = this.tiles.filter((tile) => !tile.isMatched());  
const places = loose.map((tile) => ({ x: tile.x, y: tile.y }));  
Phaser.Utls.Array.Shuffle(places);
```

Shuffle the **places**, not the tiles

16 / 20

Buttons and best results

- `Button` extends `Phaser.GameObjects.Container` (Chapter 12) - an image and a text that move as one; `onClick` is a callback
- `BestResults` - local storage (Chapter 6), checked when read back

```
if (a.stars !== b.stars) {  
    return a.stars > b.stars;  
}  
if (a.moves !== b.moves) {  
    return a.moves < b.moves;  
}  
return a.seconds < b.seconds;
```

17 / 20

Try it: the advanced version

- play both themes: which is harder, and why?
- level 3 or 4: make wrong guesses until the board shuffles
- add a fifth level - one line



18 / 20

Summary

- grids: two loops, $(COLS - 1) * GAP$, equal margins, $row * COLS + col$
- `Shuffle` for pairs; `setFrame` to turn a tile over
- rules as a **state machine**; locked states ignore the player
- flips are two tweens; **callbacks** say when they have finished
- moves and time -> stars, in one small function
- an `interface` for themes; `matchKey` vs `frame`; levels as data

19 / 20

Challenges

1. **Choose the size** - keys 1, 2, 3 for three board sizes
2. **Against the clock** - 60 seconds, counting down
3. **A hint** - H flashes a pair, for two moves
4. **Two players** - take turns; a pair means another go
5. **Triple match** - groups of three
6. **Keyboard control** - a cursor, arrows and SPACE

Next: **Chapter 14 - Catch, avoid and shoot**

20 / 20